

Sprawozdanie

Przedmiot:	Zajęcia:	Prowadzący:	Temat:	Rok i semestr:
Sieci Komputerowe	Projekt	Łukasz Maliński	Układ automatycznej regulacji działający w sieci komputerowej	2025/2026 Semestr IV

Skład sekcji			
l. p.	Imię	Nazwisko	Numer albumu
1.	Kamil	Warczek	316241
2.	Antoni	Surowy	316262

Podział obowiązków

Kamil Warczek

- Implementacja komunikacji sieciowej symulacji
- Implementacja wykrywania tracenia próbek symulacji i radzenia sobie z tym
- Implementacja protokołu sieciowego, autoryzacji i autentykacji

Antoni Surowy

- Projektowanie protokołu sieciowego
- Implementacja przesyłania synchronizacji konfiguracji
- Projektowanie GUI okienka połączeń

Protokół komunikacyjny i serializacja

Protokół komunikacyjny działa w większości przez TCP, choć ma możliwość przesyłania próbek symulacji przez dedykowany socket UDP. Nagłówki reprezentujemy przez tryb wyliczeniowy (ProtocolActions), który jest serializowany do segmentu jako pierwszy. Budowa ramek została przedstawiona *tab. 1*.

BAJTY			
0	1	1	n
ProtocolAction		Payload	

Tabela 1 Format segmentu wykorzystywanego przez protokół komunikacyjny.

Możemy wyróżnić segmenty zawierające jedynie nagłówek. Tego rodzaju segmenty służą do prostego sygnalizowania pojedynczej zmiany. Mamy także segmenty konfiguracyjne (CONFIG_*), które jako swój payload zawierają zserializowany binarnie obiekt QByteArray zawierający odpowiednią konfigurację, która jest odtwarzana na instancji, która odbiera konfigurację. Ponieważ przy komunikacji TCP zapewniona jest spójność

```
#pragma pack(push, 1)
struct SimSampleFromRegulator
{
    double wartoscZadana; // Wyjście z generatora
    double sterowanie;    // Wyjście z regulatora
    uint32_t czas;

    static SimSampleFromRegulator fromByteArray(QByteArray data)
    {
        SimSampleFromRegulator sample;
        QDataStream s(&data, QIODevice::ReadOnly);
        s >> sample.wartoscZadana >> sample.sterowanie >> sample.czas;
        return sample;
    }

    QByteArray toByteArray() const
    {
        QByteArray data;
        QDataStream s(&data, QIODevice::WriteOnly);
        s << wartoscZadana << sterowanie << czas;
        return data;
    }
};
```

Rysunek 1 Kod (de)serializacji próbki symulacji

kolejność segmentów, w samym protokole nie ma pól sumy kontrolnej, ani kolejności segmentów (za wyjątkiem datagramów z próbkami symulacji, które mogą być przesyłane po UDP). Na *rys. 1* przedstawiliśmy kod serializacji do i z QByteArray dla jednej próbki symulacji z instancji regulatora.

Nawiązywanie połączenia odbywa się, gdy nasłuchujący serwer TCP odbierze segment od klienta zawierający nagłówek żądanie połączenia (CONNECTION_REQUEST). Następnie użytkownik obsługujący instancję regulatora podejmuje decyzję o sposobie autentykacji. Wysłanie segmentu (AUTH_SUCCESS) zakańcza poprawnie autentykację użytkownika, natomiast w przypadku odrzucenia połączenia otrzymamy segment z nagłówkiem (DISCONNECT_NOTIFY), tak jak w przypadku próby rozłączenia. Ten system autentykacji pozwala na rozbudowę o różne tryby autentykacji (w programie zaimplementowaliśmy wersję zawsze akceptującą, zawsze odrzucającą i autentykację z kodem).

Zarządzanie stanem uruchomienia symulacji odbywa się poprzez segmenty z nagłówkami SIM_RUNNING i SIM_RESTART. Reset symulacji jest przekazywany sygnałem do okna głównego, aby wyczyścić pamięć wykresów na obu instancjach jednocześnie.

Zestawienie typów segmentów i odpowiadających im danych przedstawiono w *tab. 2*.

Typ segmentu	Payload
SIM_*	brak
CONNECTION_REQUEST	brak
AUTH_SUCCESS	brak
DISCONNECT_NOTIFY	brak
CONFIG_PID	zserializowana konfiguracja PID
CONFIG_ARX	zserializowana konfiguracja ARX
SAMPLE_FROM_REGULATOR	SimSampleFromRegulator
SAMPLE_FROM_ARX	SimSampleFromARX

Tabela 2 Zawartość pola payload dla poszczególnych typów segmentów protokołu.

Dla przykładu, przesyłania próbek symulacji typu SAMPLE_FROM_REGULATOR pole payload zawiera zserializowaną strukturę SimSampleFromRegulator, której pola przedstawiono w *tab. 3*.

wartoscZadana : double (8B)	sterowanie : double (8B)	czas : uint32_t (4B)
-----------------------------	--------------------------	----------------------

Tabela 3 Struktura danych SimSampleFromRegulator wykorzystywana jako zawartość pola payload komunikatu SAMPLE_FROM_REGULATOR.

Wykrywanie gubienia pakietów

Status

● Połączono

IP lokalne: 10.0.0.173

IP zdalne: 127.0.0.1

TX: 30

RX: 22

Rysunek 2 Okno GUI -
Pakiety

Wykrywanie gubienia pakietów zrealizowaliśmy poprzez przesyłanie czasu próbki symulacji w milisekundach od jej początku w segmencie przesyłanym po sieci, a podczas odbioru segmentu porównujemy, czy czas w odebranej próbce odpowiada naszym oczekiwaniom tego, że będzie odpowiadać obecnemu czasowi symulacji w milisekundach + interwał (jeśli jest wcześniejszy, to ten segment przyszedł z opóźnieniem).

Ponieważ bezpośrednio nie operujemy na pakietach, nie interesuje nas liczba wystanych/odebranych pakietów, a segmentów więc tą liczbę mierzymy i wypisujemy w oknie konfiguracji połączenia sieciowego jako RX/TX. Dzięki temu możemy zobaczyć, ile segmentów zostało odrzuconych porównując te dwie liczby. Zostało to pokazane na rys. 2.

Jeśli próbka z instancji obiektu ARX nie zostanie odebrana i przetworzona przez instancję regulatora na czas (przed następnym wywołaniem symulacji przez timer interwału), to na wykresy przekazujemy kolejny element, którego wartości są takie same jak poprzedniego dodanego, ale zawierający oznaczenie, że ta próbka jest interpolowana (czarnym okręgiem). Wygląd takiego oznaczenia przedstawiliśmy na rys. 3. Dodatkowo w momencie wykrycia takiej sytuacji kontrolka interwału w GUI zmienia kolor tła na czerwony do czasu otrzymania brakującej próbki.



Rysunek 3 Przedstawienie zgubionych pakietów na wykresie

Historia rozwoju aplikacji

1. Zapoznaliśmy się z otrzymanym kodem.
2. Wprowadziliśmy poprawki do kodu, które poniosły czytelność, wydajność symulacji oraz naprawiły błędy i odstępstwa od specyfikacji.
3. Dodaliśmy okno do zarządzania połączeniem sieciowym do aplikacji.
4. Stworzyliśmy definicję protokołu sieciowego, budowę segmentów i wybraliśmy sposób serializacji danych konfiguracyjnych (binarny).
5. Zaimplementowaliśmy funkcjonalności sieciowe do osobnej klasy odpowiadającej za komunikację siecią (łączenie i autoryzacja).
6. Zaimplementowaliśmy przesyłanie konfiguracji symulacji po sieci.
7. Poprawiliśmy mechanizm rysowania wykresów na napisany samodzielnie, bardziej optymalny sposób, który pozwala na optymalizacje specyficzne dla rysowanych wykresów.
8. Dodaliśmy funkcjonalność przesyłania próbek symulacji po sieci przez TCP i aktualizację elementów symulacji po obu stronach, aby zachować ciągłość po przejściu w tryb lokalny.
9. Zintegrowaliśmy kod klasy odpowiedzialnej za komunikację siecią z warstwą usług.
10. Dodaliśmy opcję przesyłania próbek symulacji po UDP
11. Zmieniliśmy sposób serializacji próbek symulacji (od teraz przesyłamy też czas)
12. Dodaliśmy mechanizm ekstrapolacji próbek, dla próbek które nie zdążyły przejść przez połączenie sieciowe na czas (poprzez podtrzymanie ostatniej wartości).

Napotkane trudności

Problem autentyfikacji i autoryzacji

Jednym z napotkanych podczas projektowania protokołu sieciowego problemów, okazał się problem autoryzacji i autentykacji użytkownika. Wynikał on z zapisu o zabezpieczeniu w jakiś sposób przed przypadkowym podłączeniem do drugiej instancji programu. Rozwiązaliśmy go poprzez implementację systemu uwierzytelnienia polegającego na przesłaniu żądania połączenia przez klienta, następnie serwer decyduje o podłączeniu. Jeśli użytkownik zażąda połączenia bez autentykacji, mamy możliwość połączenia bez lub z użyciem 4 cyfrowego kodu. Autoryzacja jest rozwiązana poprzez sprawdzenie czy dany użytkownik jest autoryzowany do komunikacji, jeśli nie jest, to konfiguracja i próbki symulacji są odrzucane na etapie czytania zawartości segmentu/datagramu. Zarówno dla próbek przesyłanych po TCP jak i po UDP.

Utrzymanie symulacji podczas tracenia segmentów

Problem utrzymania symulacji podczas tracenia segmentów, został rozwiązany poprzez ekstrapolację wartości z obiektu ARX, gdy odpowiedź od niego nie przyjdzie na czas ostatnią wartością jaką odpowiedział. Pozwala to na utrzymanie stałego interwału symulacji po stronie instancji regulatora. Jeśli natomiast klient nie odpowie przez 10 sekund, połączenie jest rozłączane i symulacja trwa nadal w trybie lokalnym na obu instancjach.

Przesyłanie konfiguracji w sposób nieobciążający symulacji

Problem ten został rozwiązany poprzez podzielenie konfiguracji na osobne segmenty (o typie segmentu decyduje jego nagłówek) dla każdego obiektu. Dzięki temu przesyłane pakiety zawierające nowe parametry konfiguracji są relatywnie małe, nie opóźniają symulacji a zatwierdzenie nowych wartości parametrów nie zajmuje wiele czasu.

Synchronizacja czasowa symulacji między instancjami

Problemem, który napotkaliśmy podczas przesyłania próbek symulacji po UDP była synchronizacja czasowa symulacji. Problem ten rozwiązaliśmy poprzez przesyłanie czasu w postaci wartości w milisekundach od początku symulacji, która to wartość jest wykorzystywana do sprawdzenia czy dana próbka jest najnowszą próbką, czy starą próbką, która dopiero teraz dotarła po sieci i powinna być odrzucona.

Czego się nauczyliśmy

Kamil Warczek

Projekt ten nauczył mnie implementacji protokołu sieciowego w niskopoziomowym języku programowania, a także weryfikowania działania mechanizmów autoryzacji i autentykacji.

Praca na kodzie źródłowym innych osób pokazała jak ważne jest utrzymanie przejrzystości kodu i/lub aktualnej dokumentacji. Pozwoliła mi także zobaczyć jak ten sam problem, który rozwiązywaliśmy w zeszłym semestrze można rozwiązać inaczej.

Dodatkowo poznałem zasadę działania filtra przeciwnasyceniowego, który był zadaniem przeznaczonym jedynie dla sekcji 3 osobowych, ale aby zweryfikować poprawność jego działania musiałem się nauczyć, jak powinien on funkcjonować w regulatorze PID.

Antoni Surowy

Podczas realizacji projektu nauczyłem się projektować strukturę segmentów protokołu komunikacyjnego, dobierać sposób serializacji danych oraz tworzyć rozwiązania umożliwiające łatwe rozszerzanie protokołu o nowe typy komunikatów.

Implementacja synchronizacji konfiguracji pozwoliła mi lepiej zrozumieć zagadnienia związane z przesyłaniem danych pomiędzy współpracującymi instancjami programu oraz różnice pomiędzy wykorzystaniem protokołów TCP i UDP.

Projekt pozwolił mi również zdobyć doświadczenie w pracy z kodem napisanym przez inne osoby, dzięki czemu przekonałem się, jak ważne są czytelna architektura programu, spójny styl programowania oraz aktualna i czytelna dokumentacja.

Dodatkowo poznałem praktyczne zastosowanie mechanizmów uwierzytelniania i autoryzacji połączeń sieciowych.